“VIBECHECK ROOMS”

MINOR PROJECT (BCA-508)

SUBMITTED TO

Chaudhary Charan Singh University, Meerut

For the partial fulfillment of the degree of the Bachelor’s in Computer Application (2024-27).



UNDER THE GUIDANCE OF

Prof. _____

IMS NOIDA

SUBMITTED BY

GAURAV SINGH

University Roll No. 240919106084

INSTITUTE OF MANAGEMENT STUDIES, NOIDA

PROFORMA FOR APPROVAL OF BCA MINOR PROJECT (BCA-508)

Roll no. **240919106084**

Name of the Student: **Gaurav Singh**

E-mail: **gauravsingh.bcr24079@imsnoida.com**

Mob. No: **+91 7289860248**

Title of the MINOR Project: **VIBECHECK ROOMS**

Name of the Guide:

For Office Use Only:

		Signature of the Mentor
		Date:
Approved	Not Approved	

Suggestions (if any):-

1.
2.
3.
4.
5.

ACKNOWLEDGMENT

I would like to express my sincere thanks to HOD SOIT **Ms. Monika Dixit Bajpai** for her valuable guidance and support in completing my project titled "**VIBECHECK ROOMS**", and I would also like to express my gratitude to the entire faculty of SOIT for their support and suggestions.

I am very grateful to my Minor Project (BCA-508) mentor **Prof.** _____ for giving valuable time and constructive guidance in preparing the Synopsis and the Minor Project (BCA-508). It would not have been possible to complete this project in such a short period of time without the kind encouragement and valuable guidance received from my mentor.

The successful completion of my Minor Project would not have been possible without the dedicated support of all my mentors, the faculty members of the School of Information Technology, and my friends.

I am also thankful to my classmates and peers who helped me in the successful completion of this project.

Signature .....
GAURAV SINGH
University Roll No.: 240919106084

CERTIFICATE OF ORIGINALITY

I hereby declare that the Minor Project (BCA-508) titled "**VIBECHECK ROOMS**" has been submitted to the School of IT, IMS Noida, which is affiliated with **Chaudhary Charan Singh University, Meerut (U.P.)**, for the partial fulfillment of the degree of Bachelor of Computer Application, in A.Y. 2026 (Batch 2024-27). This is my original work and it has not previously formed the basis for the award of any other degree, diploma or other title from any other University.

Project Guide

Prof. _____

Department of SOIT

IMS Noida

Student

Gaurav Singh

BCA

University Roll No. 240919106084

Date: .....

Place: IMS Noida

INTERNSHIP COMPLETION CERTIFICATE



5

TABLE OF CONTENTS

S.No	Title	Page No.
	ACKNOWLEDGMENT	3
	CERTIFICATE OF ORIGINALITY	4
1	**INTRODUCTION / OVERVIEW OF THE PROJECT** 1.1 Introduction 1.2 Problem Background 1.3 Real-World Relevance 1.4 Solution Overview 1.5 Objectives of the Project 1.6 Scope of the Project	7-9
2	**PERFORMANCE OF THE PROJECT** 2.1 Feasibility Study (Technical, Operational, Economic, Scheduling) 2.2 Effectiveness 2.3 Operational Benefits 2.4 Performance Targets, Capacity and Latency 2.5 Scalability	10-13
3	**PROJECT ARCHITECTURE** 3.1 Three-Tier Architecture Overview 3.2 Presentation Tier 3.3 Application / API Tier 3.4 Data Tier 3.5 Deployment and Security Design	14-16
4	**PROJECT FLOW** 4.1 End-to-End Flowchart 4.2 User Journey Walkthrough 4.3 Core Business Transaction: Mood Check-in 4.4 Vibe Score Logic 4.5 Validation Rules	17-20
5	**DATA FLOW DIAGRAMS (DFD)** 5.1 Level 0 DFD (Context Diagram) 5.2 Level 1 DFD 5.3 Level 2 DFD (Mood Check-in Process)	21-24
6	**USE CASE DIAGRAM** 6.1 Use Case Diagram 6.2 Actors and Use Case Descriptions	25-26

S.No	Title	Page No.
7	**ENTITY RELATIONSHIP DIAGRAM (ERD)** 7.1 ER Diagram 7.2 Table Structure (PK, FK, Attributes) 7.3 Relationships and Cardinality 7.4 Data Integrity Rules	27-29
8	**HARDWARE AND SOFTWARE REQUIREMENTS** 8.1 Hardware Requirements 8.2 Software Requirements 8.3 Deployment Hosts	30-31
9	**FUTURE SCOPE**	32
10	**LIMITATIONS OF THE PROJECT**	33
11	**FRONT-END PAGES** 11.1 Landing / Home Page 11.2 Login / Registration Modal 11.3 Room Dashboard (Main User Interaction) 11.4 Admin Management and Record Control Panel	34-37

1. INTRODUCTION / OVERVIEW OF THE PROJECT

1.1 Introduction

VibeCheck Rooms is a full-stack web application that lets a small group of friends share how they feel, and the music that matches that feeling, inside a private shared space called a **room**. A member creates a room, receives a six-character room code, and invites friends to join with that code. Every member can then post a **mood check-in** made of a mood chosen from eight options, an energy level from 1 to 5, an optional short note of up to 80 characters, and up to three songs (title, artist and an optional link).

From these check-ins the system calculates a **Group Vibe Score** between 0 and 100, finds the dominant mood of the room, draws a mood-mix chart and builds a **shared playlist** from the songs added by all members. A room administrator can moderate the room by removing members, deleting check-ins, regenerating the room code or closing the room.

The application follows the standard three-tier architecture: a React front end hosted on Vercel, a Node.js and Express REST API hosted on Render, and a PostgreSQL database with authentication provided by Supabase. The complete stack runs on free plans, so the project can be deployed to a public HTTPS link at zero cost.

1.2 Problem Background

Friend groups today coordinate mostly through chat applications. A mood or a song recommendation posted in a busy group chat is quickly buried under newer messages, and there is no simple way to see at a glance how the group is feeling right now. Music is one of the strongest social connectors for young people: according to figures published by Spotify in 2022, more than 11 million user-generated playlists on its platform carried the word “friend” in their title, and Gen Z listeners were the age group that streamed such playlists the most.

Existing social music features address only part of the need. Spotify Blend, for example, merges the listening tastes of up to ten friends into one automatically generated playlist, but it depends on the listening history of a single proprietary platform and does not capture how people feel on a particular day. There is no lightweight, platform-independent space where friends can combine a self-reported mood with a few songs and instantly see the combined vibe of the group.

Problem statement: to design and develop a free, easy-to-use web application in which friends can create private rooms, post daily mood check-ins with songs, and view a transparent, formula-based group vibe summary and shared playlist, supported by simple role-based moderation.

1.3 Real-World Relevance

- **Friend groups and hostel or PG roommates** can use a room as a daily, low-effort way to stay aware of each other’s mood.

- **Long-distance friends and families** can keep a light emotional connection without having to schedule calls.
- **College clubs, project teams and classes** can use a room as an informal team-spirit check.
- **Music communities** get a simple way to build a shared playlist from everyone's current mood.
- **Learning value:** the project demonstrates authentication, relational database design, REST APIs, business logic, role-based access control and cloud deployment in one small system.

VibeCheck Rooms is a social and entertainment tool. The vibe score is a simple, self-reported indicator and is not a medical or psychological measurement.

1.4 Solution Overview

Feature	Description
Account and login	Email and password sign-up and login through Supabase Auth. Sessions are handled with JWT tokens.
Rooms	Create a room and get a unique 6-character code. Join with a code. Member list with roles (Admin or Member).
Mood check-in	Choose one of 8 moods, an energy level (1 to 5), an optional note, and add up to 3 songs.
Group Vibe Score	Average mood score of each member's latest check-in in the last 24 hours, shown with a label and the dominant mood.
Mood mix chart	Bar chart showing how many members chose each mood.
Shared playlist	All songs added by members, newest first, with the name of the member who added each song.
Admin panel	Remove members, delete check-ins, regenerate the room code, close the room and view room statistics.
Auto refresh	The dashboard refreshes automatically every 20 seconds and also has a manual Refresh button.

1.5 Objectives of the Project

1. To build a secure web application in which friends can create and join private rooms using a room code.
2. To let every member share a mood, an energy level and up to three songs in a few seconds.
3. To calculate a transparent Group Vibe Score and dominant mood from stored check-ins.
4. To generate a shared playlist and a mood-mix chart from the data of all members.
5. To provide role-based access so that only room admins can moderate members and records.

6. To store all data permanently in a cloud PostgreSQL database with proper keys and constraints.
7. To deploy the complete system on a public HTTPS URL using free-tier services, keeping the total cost at Rs. 0.

1.6 Scope of the Project

In scope	Out of scope (listed under Future Scope)
Registration, login and logout Create and join rooms with a code Mood check-ins with up to 3 songs Group Vibe Score, dominant mood and mood-mix chart Shared playlist Admin moderation panel Responsive user interface Live deployment on free cloud services	Audio streaming or music-service API integration Real-time WebSocket updates AI-based mood analysis or recommendations Native mobile application Payments of any kind Push notifications In-app chat

2. PERFORMANCE OF THE PROJECT

2.1 Feasibility Study

2.1.1 Technical Feasibility

The project uses mature, open-source technologies that are widely documented: React and Vite for the user interface, Tailwind CSS for styling, Node.js and Express for the API, and PostgreSQL for the database. Authentication is delegated to Supabase Auth, so passwords never have to be handled manually. The developer works in JavaScript on both the client and the server, so a single language covers the whole stack. The hosting platforms (Vercel, Render and Supabase) all provide free plans that comfortably support a small project, so the project is technically feasible.

Component	Technology	Why it is suitable
Front end	React 18, Vite, Tailwind CSS	Component-based UI, fast builds, quick responsive styling
API	Node.js 20, Express	Simple REST routing and middleware, same language as the front end
Database	PostgreSQL (Supabase)	Relational model with keys, constraints and joins
Authentication	Supabase Auth (JWT)	Secure registration, login and sessions without custom password code
Deployment	Vercel, Render, GitHub	Free plans with automatic deployment from GitHub

2.1.2 Operational Feasibility

- The interface has only three main actions (create a room, post a check-in, read the vibe) and needs no user training.
- Admin tasks are limited to a single panel with clearly marked controls.
- Seeded demonstration data lets the system be shown immediately, even to a first-time viewer.
- All services are managed in the cloud, so day-to-day maintenance is minimal.

2.1.3 Economic Feasibility

The project is designed to cost nothing. Every service used has a free plan that is sufficient for the expected load, and no paid API, domain or software licence is required.

Item	Service used	Cost (Rs.)
Front-end hosting	Vercel Hobby plan	0
API hosting	Render Free web service	0
Database and authentication	Supabase Free plan	0
Source control	GitHub Free	0

Item	Service used	Cost (Rs.)
Domain name	Default vercel.app sub-domain	0
Development tools	VS Code, Node.js, Git (open source)	0
Total		**0**

Free plans can change over time. The architecture uses standard components, so any layer can be moved to another provider if needed.

2.1.4 Scheduling Feasibility

The official department plan allows coding and development from 7 October 2026 to 23 October 2026, a first live review on 24 October 2026 and final submission on 31 October 2026. The work is divided into small phases so that the deployed application is ready before the review date.

Date (2026)	Planned work
7 - 8 Oct	Repository, front-end and API skeleton, database schema and seed data
9 - 10 Oct	Authentication, protected routes, create and join room
11 - 13 Oct	Mood check-in form, songs, validation rules
14 - 15 Oct	Vibe engine, mood-mix chart, shared playlist, dashboard
16 - 17 Oct	Admin panel, UI polish, responsive layout
18 - 19 Oct	Testing (positive, negative and role tests) and bug fixing
20 Oct	Deployment on Vercel, Render and Supabase
21 - 22 Oct	Live end-to-end testing, screenshots, documentation
23 Oct	Code freeze and demo rehearsal
24 Oct	**First live project review (deployed link)**
31 Oct	Final project presentation and hard copy file submission

2.2 Effectiveness

- **Speed:** posting a check-in takes a few taps (mood, energy and optional songs) and is designed to take under 15 seconds.
- **Clarity:** one number and one label summarise the whole room, so nobody has to scroll through old chat messages.
- **Transparency:** the score uses a fixed, published formula, and every member can see how it is computed.
- **Engagement:** a playlist built from real check-ins gives members a reason to come back every day.
- **Consistency:** all data lives in PostgreSQL, so the same room state is visible from any device or browser.

2.3 Operational Benefits

- **Zero cost:** no hosting, database or licence fees.
- **No installation:** the application runs in a normal web browser on laptops and phones.
- **Role-based control:** admins can moderate a room without needing access to the database.
- **Low maintenance:** managed cloud services handle servers, backups of the platform and HTTPS certificates.
- **Easy demonstration:** a seeded demo room shows every feature within a few minutes.

2.4 Performance Targets, Capacity and Latency

The table lists the design targets for a free-tier deployment. These values will be verified during the testing phase.

Parameter	Target	Remarks
Landing page load	Under 2.5 seconds on normal 4G or broadband	Static files are served from the Vercel CDN
API response (server awake)	Under 800 ms for a check-in, under 500 ms for dashboard data	Indexed queries on foreign-key columns
First request after idle	Up to about 1 minute	A Render free web service spins down after 15 minutes without traffic. A free uptime monitor can call /api/health every 5 minutes to keep it awake
Dashboard refresh	Automatic every 20 seconds	Polling is used instead of WebSockets to keep the system simple
Room size	2 to 10 members recommended	Keeps the vibe score and playlist meaningful
Concurrent users	Tens of simultaneous users	Enough for demonstration and small friend groups
Database capacity	500 MB (Supabase free plan)	A check-in with songs needs only a few hundred bytes, so the limit is far above project needs
Registered users	Up to 50,000 monthly active users	Supabase free-plan authentication allowance

Sources: Render and Supabase plan documentation, checked October 2026. Providers may change these limits.

2.5 Scalability

- **Stateless API:** the Express server keeps no session data in memory, so more instances can be added behind a load balancer when needed.
- **Plan upgrade path:** moving the API to a paid Render instance removes the idle spin-down, and a paid Supabase plan removes the inactivity pause and raises the storage

limit.

- **Database tuning:** indexes on room_id, user_id and created_at, plus pagination of check-in and song history.
- **Caching:** the vibe score can be cached per room for a few seconds to reduce repeated calculations.
- **Push updates:** Supabase Realtime can replace 20-second polling when instant updates are required.

3. PROJECT ARCHITECTURE

3.1 Three-Tier Architecture Overview

VibeCheck Rooms is built as a three-tier system. Each tier has one clear responsibility and talks only to the tier next to it, which makes the system easier to develop, test, secure and scale.

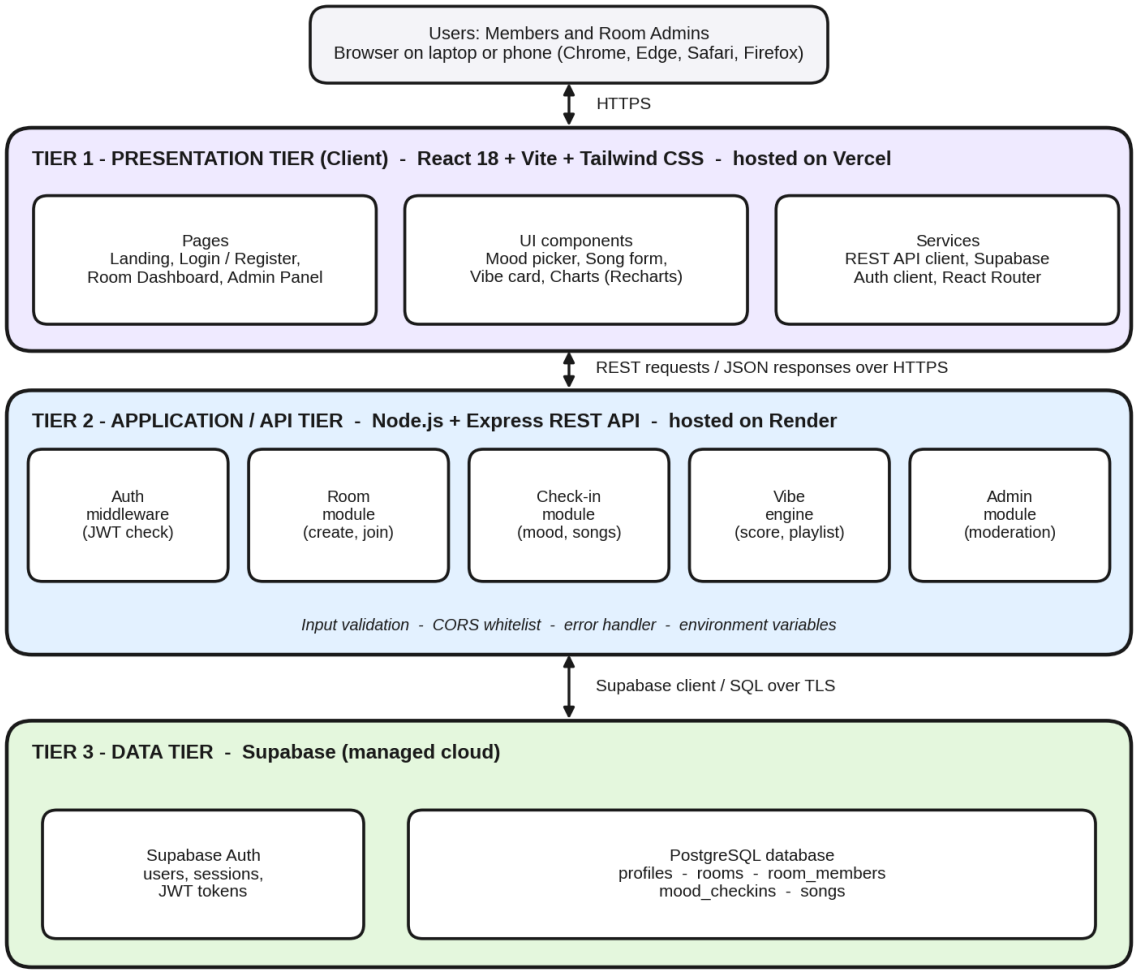


Figure 3.1: Three-tier architecture of VibeCheck Rooms

3.2 Presentation Tier (Client)

The presentation tier is a single-page application written in React 18 and built with Vite. Tailwind CSS provides the responsive dark-theme styling, React Router handles navigation between the landing page, login, room dashboard and admin panel, Recharts draws the mood-mix chart and Lucide supplies the icons. The front end never holds database credentials; it talks to the API over HTTPS and uses the Supabase client only for sign-in and session handling. The tier is hosted on Vercel and refreshes room data every 20 seconds.

3.3 Application / API Tier

The application tier is a Node.js 20 and Express REST API hosted on Render. It contains five modules: authentication middleware (verifies the JWT on each request), room module, check-in module, vibe engine and admin module. Shared middleware performs input validation, CORS filtering and error handling. The tier contains all business rules, such as the vibe score formula, the three-song limit and room-membership checks.

Method	Endpoint	Purpose	Access
GET	`/api/health`	Deployment health check	Public
POST	`/api/rooms`	Create a room and generate its 6-character code	Member
POST	`/api/rooms/join`	Join a room using a room code	Member
GET	`/api/rooms/me`	List the rooms of the logged-in user	Member
GET	`/api/rooms/:id`	Room details and member list	Room member
POST	`/api/rooms/:id/checkins`	Post a mood check-in with up to 3 songs	Room member
GET	`/api/rooms/:id/vibe`	Group vibe score, dominant mood, mood mix	Room member
GET	`/api/rooms/:id/playlist`	Shared playlist of the room	Room member
DELETE	`/api/checkins/:id`	Delete a check-in (own, or any as admin)	Owner or Admin
DELETE	`/api/rooms/:id/members/:userId`	Remove a member from the room	Admin
POST	`/api/rooms/:id/regenerate-code`	Create a new room code	Admin
PATCH	`/api/rooms/:id/close`	Close (archive) the room	Admin
GET	`/api/rooms/:id/stats`	Room statistics for the admin panel	Admin

3.4 Data Tier

The data tier is a managed Supabase project. Supabase Auth stores user accounts and issues JWT tokens, while PostgreSQL stores five application tables: **profiles, rooms, room_members, mood_checkins and songs**. Primary keys, foreign keys, unique constraints and check constraints (for example energy between 1 and 5) enforce data integrity inside the database itself. Indexes on foreign-key columns keep queries fast, and Row Level Security is enabled as a second line of defence.

3.5 Deployment and Security Design

- **Continuous deployment:** pushing code to the GitHub main branch automatically builds and publishes the front end on Vercel and the API on Render.
- **HTTPS everywhere:** all traffic between browser, API and database is encrypted.
- **No password handling in the app:** credentials are managed by Supabase Auth.
- **Authentication and authorisation:** the JWT is verified on every protected route, room membership is checked before any read or write, and admin-only routes also check the role stored in room_members.
- **Secrets:** keys are kept in environment variables. The service-role key exists only on the API server, never in the front end, and .env files are never committed to Git.
- **Request hygiene:** strict input validation, a CORS whitelist containing only the Vercel URL, and generic error messages that do not expose internal details.

4. PROJECT FLOW

4.1 End-to-End Flowchart

The flowchart shows the complete journey from the landing page to the core transaction (posting a mood check-in) and the optional admin actions.

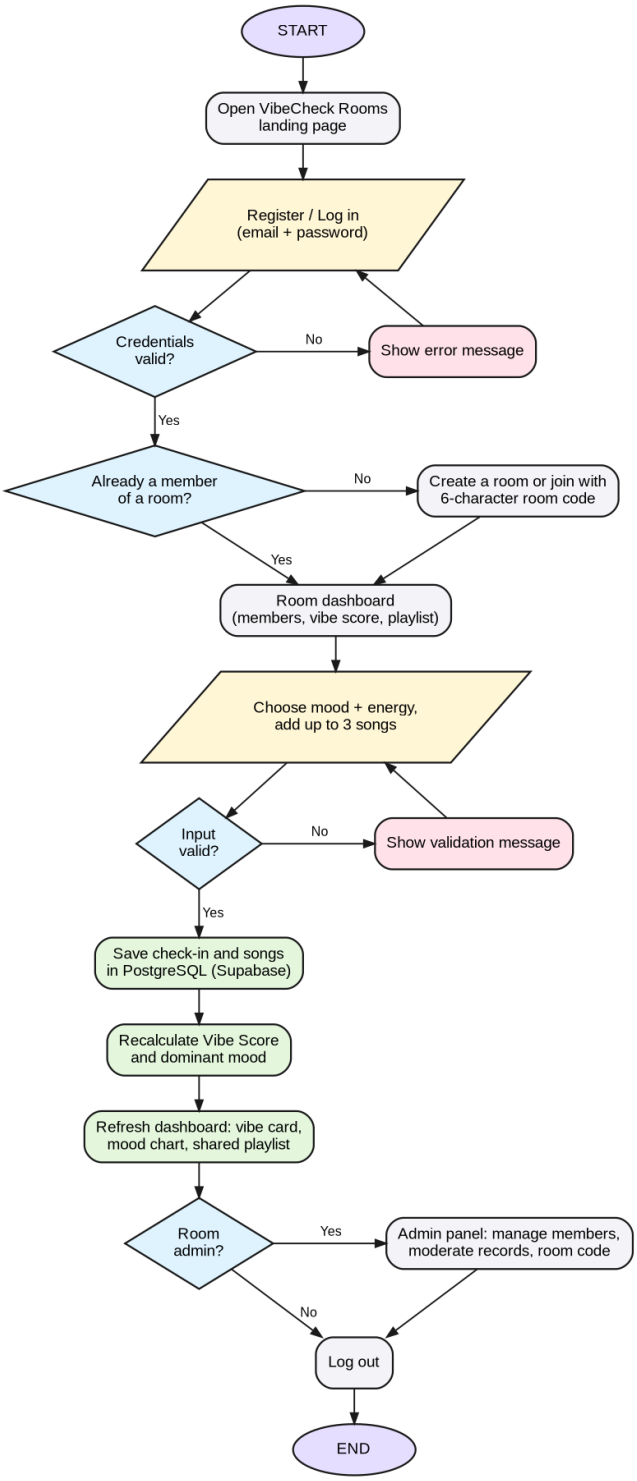


Figure 4.1: Project flow from landing page to vibe score update

4.2 User Journey Walkthrough

1. The user opens the landing page and chooses **Get started** or **Log in**.
2. The user registers with name, email and password, or logs in. Supabase Auth validates the credentials and returns a session token.
3. A user who is not yet in a room either creates a room (and receives a 6-character code) or joins a friend's room by entering the code.
4. The room dashboard opens and shows the latest moods of members, the Group Vibe Score, the mood mix and the shared playlist.
5. The user selects a mood, sets an energy level from 1 to 5, optionally writes a short note and adds up to three songs.
6. The front end checks the input and sends a POST request to the API. The API verifies the token and the room membership.
7. The API saves the check-in and the song rows in PostgreSQL.
8. The vibe engine recalculates the Group Vibe Score and the dominant mood from the latest check-ins.
9. The dashboard refreshes with the new score, chart and playlist. Other members see the update within 20 seconds.
10. A room admin can open the admin panel to remove members, delete records, regenerate the room code or close the room.
11. The user logs out and the session is cleared.

4.3 Core Business Transaction: Mood Check-in

Step	Action	Tier
1	User chooses mood, energy, optional note and up to 3 songs	Presentation
2	Form validation and POST /api/rooms/:id/checkins with the session token	Presentation
3	JWT verified; room membership and room status (active) checked	Application
4	Input validated against the rules in Section 4.5	Application
5	One row inserted in mood_checkins and up to 3 rows in songs (single transaction)	Data
6	Vibe engine reads the latest check-in of each member and recalculates the score	Application / Data
7	Updated vibe score, mood mix and playlist returned and displayed	Presentation

4.4 Vibe Score Logic

Every mood has a fixed score between 0 and 100 stored in the API configuration. These scores are simple values chosen by the developer for this application and are not a scientific measure.

Mood	Score	Mood	Score
Happy	90	Focused	65
Hyped	85	Nostalgic	55
Romantic	80	Stressed	30
Chill	70	Sad	20

Group Vibe Score = round(sum of mood scores of every member's latest check-in in the last 24 hours ÷ number of members who checked in).
Dominant mood = the mood chosen by the most members (if there is a tie, the more recent check-in wins).
Average energy = mean of the energy values of those check-ins.

Score range	Label
0 - 39	Low vibes
40 - 64	Balanced vibes
65 - 84	Good vibes
85 - 100	Peak vibes

Worked example (room "Late Night Crew"):

Member	Mood	Energy	Mood score
Rahul	Chill	3	70
Aman	Hyped	5	85
Rohit	Happy	4	90
Gaurav	Chill	3	70
Result	Dominant: Chill (2 of 4)	Average: 15 ÷ 4 = 3.8	Total 315, 315 ÷ 4 = 78.75, so **Vibe Score = 79 (Good vibes)**

4.5 Validation Rules

Field	Rule	Error response
mood	Must be one of the 8 moods	400 Bad Request
energy	Whole number from 1 to 5	400 Bad Request
note	Optional, maximum 80 characters	400 Bad Request
songs	0 to 3 per check-in; title and artist required (maximum 100 characters each)	400 Bad Request
link	Optional; must start with http:// or https://	400 Bad Request
room code	Exactly 6 letters or digits; room must exist and be active	404 Not Found
membership	A user cannot join the same room twice	409 Conflict
authentication	Valid session token required on protected routes	401 Unauthorized
authorisation	Must belong to the room; admin role needed for moderation routes	403 Forbidden

5. DATA FLOW DIAGRAMS (DFD)

5.1 Level 0 DFD (Context Diagram)

The context diagram shows the whole system as one process that exchanges data with two external entities: the **Member** and the **Room Admin**.

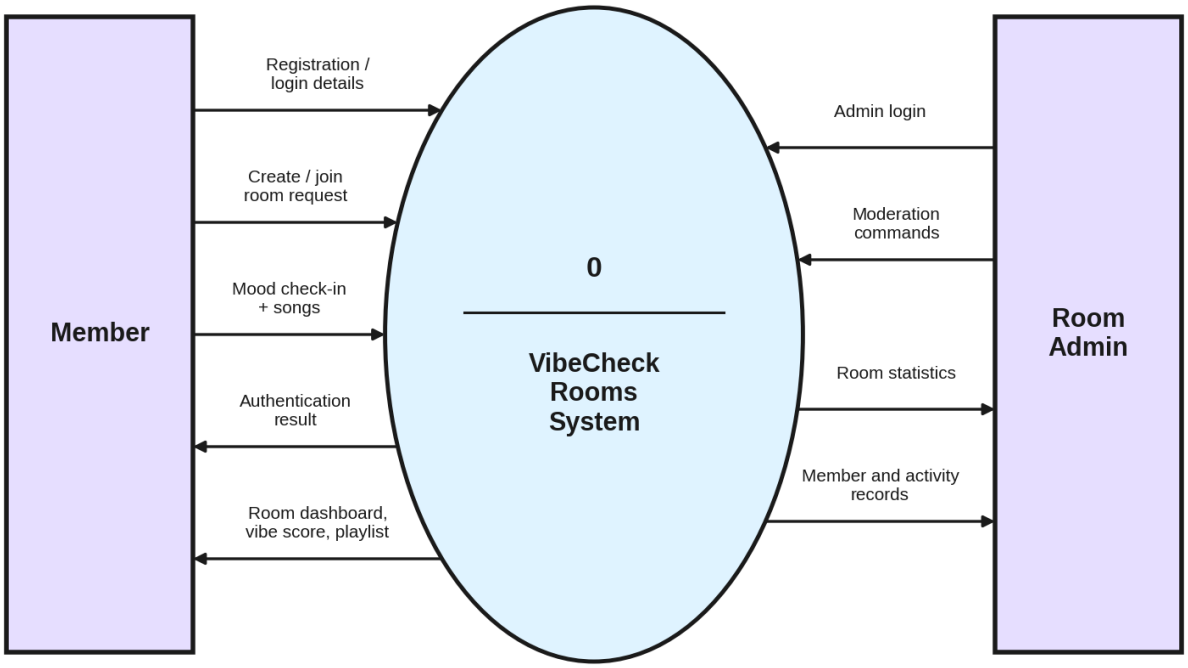


Figure 5.1: Level 0 DFD (Context Diagram)

Entity	Data sent to the system	Data received from the system
Member	Registration and login details, create or join room request, mood check-in with songs	Authentication result, room dashboard, vibe score and shared playlist
Room Admin	Admin login, moderation commands (remove member, delete check-in, regenerate code)	Room statistics, member and activity records

5.2 Level 1 DFD

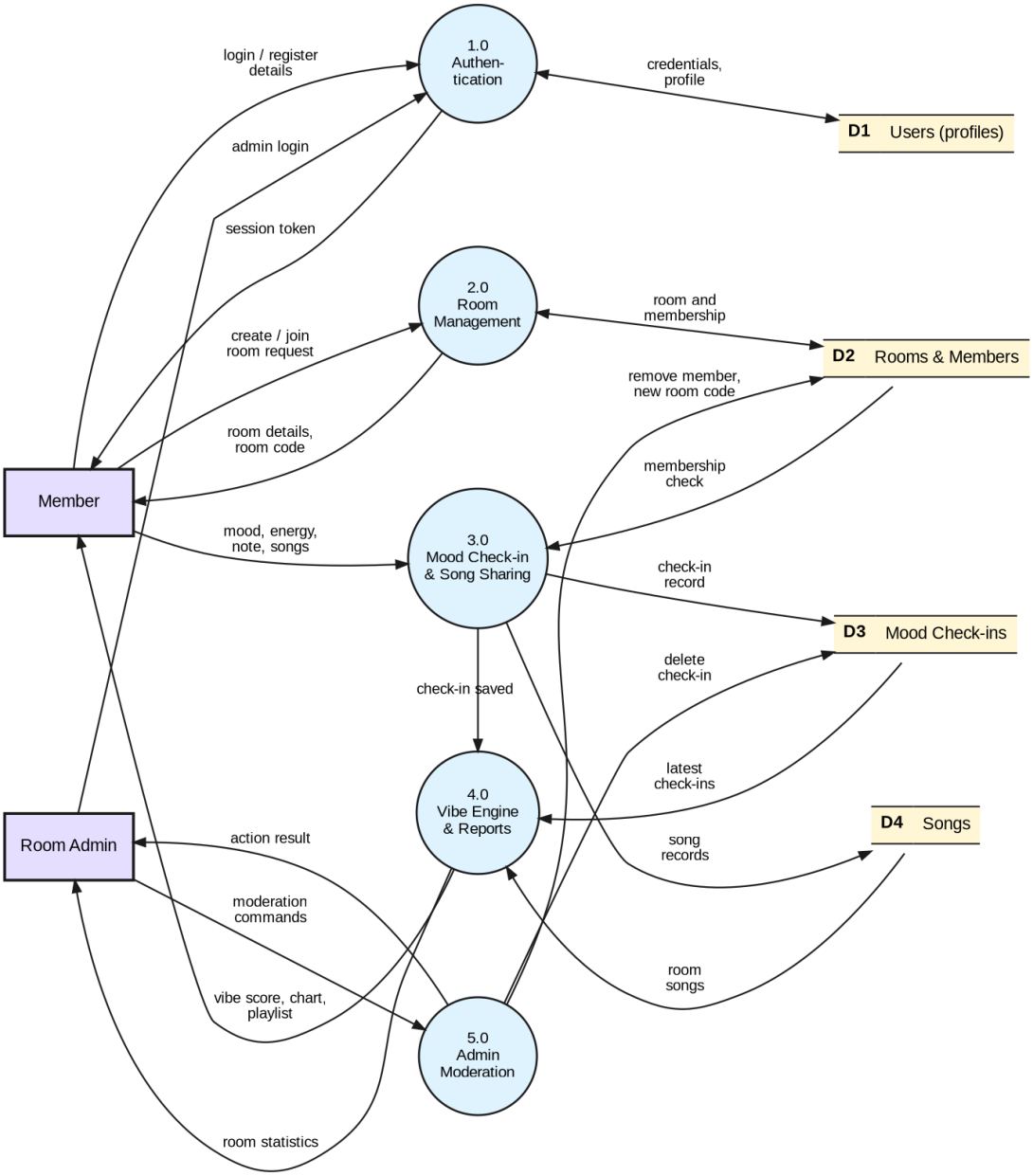


Figure 5.2: Level 1 DFD of VibeCheck Rooms

Process	Function	Data stores used
1.0 Authentication	Registers and logs in users and issues the session token	D1 Users (profiles)
2.0 Room Management	Creates rooms, joins rooms by code, lists members	D2 Rooms and Members
3.0 Mood Check-in and Song Sharing	Validates and stores check-ins and songs (core processing)	D2, D3 Mood Check-ins, D4 Songs
4.0 Vibe Engine and Reports	Calculates vibe score, mood mix, playlist and statistics	D3, D4

Process	Function	Data stores used
5.0 Admin Moderation	Removes members, deletes check-ins, regenerates room code	D2, D3

5.3 Level 2 DFD (Process 3.0: Mood Check-in and Song Sharing)

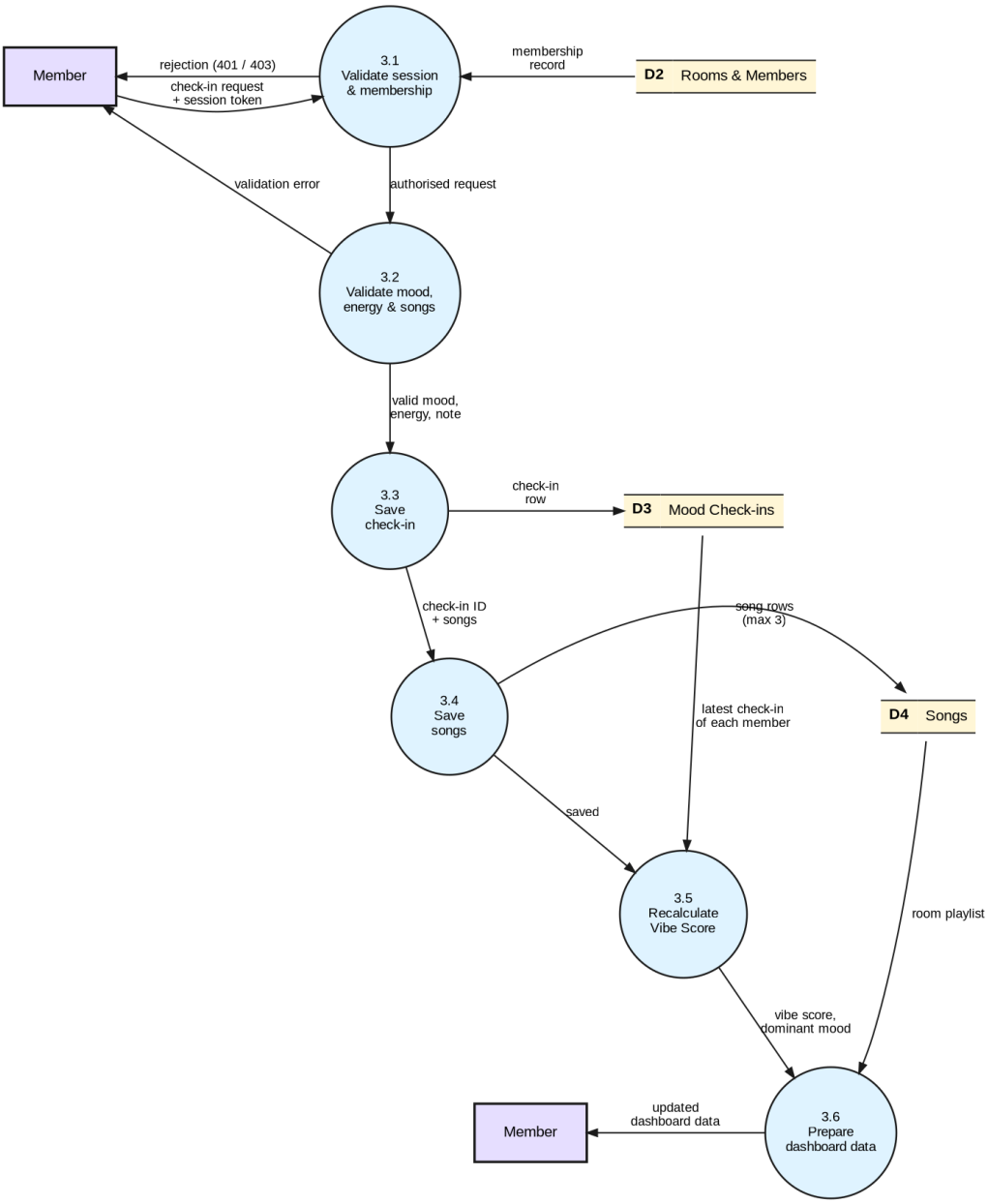


Figure 5.3: Level 2 DFD of the mood check-in transaction

Sub-process	Description
3.1 Validate session and membership	Checks the token and confirms in D2 that the user belongs to the room
3.2 Validate mood, energy and songs	Applies the validation rules and returns an error if the data is invalid
3.3 / 3.4 Save check-in and songs	Writes one row to D3 and up to three rows to D4
3.5 Recalculate Vibe Score	Reads the latest check-in of each member from D3 and computes the score
3.6 Prepare dashboard data	Combines score, mood mix and playlist (from D4) and returns them to the member

6. USE CASE DIAGRAM

6.1 Use Case Diagram

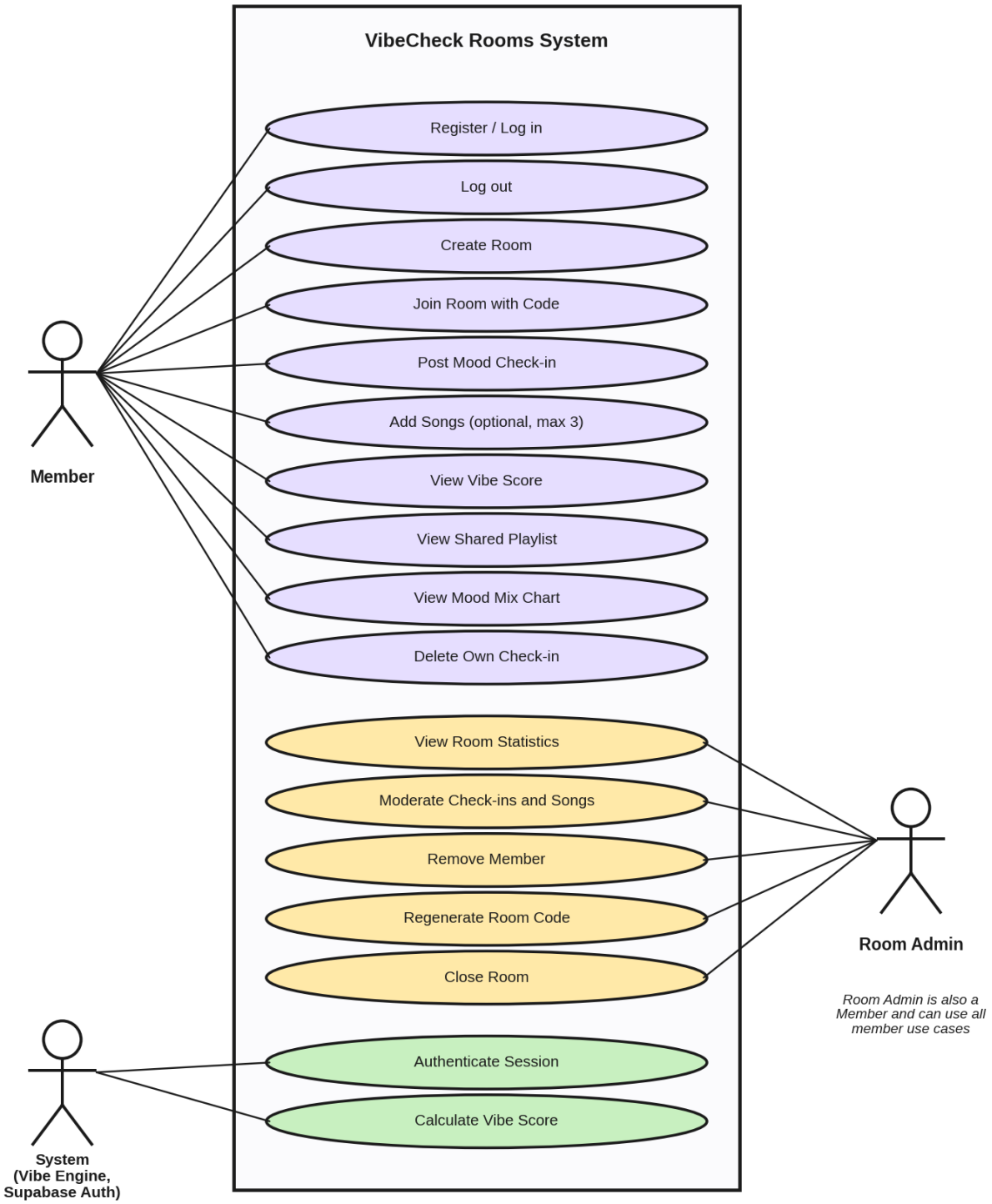


Figure 6.1: Use case diagram of VibeCheck Rooms

6.2 Actors and Use Case Descriptions

Actor	Description
Member	A registered user who belongs to one or more rooms. Can post check-ins and view the vibe score, mood mix and playlist.
Room Admin	The creator of a room. A Room Admin is also a Member and can additionally moderate the room.
System	The Vibe Engine in the API and Supabase Auth, which authenticate sessions and calculate the vibe score automatically.

Use case	Actor	Description
Register / Log in	Member	Create an account or sign in with email and password
Log out	Member	End the current session
Create Room	Member	Create a room and receive a 6-character room code
Join Room with Code	Member	Enter a friend’s code to become a member of that room
Post Mood Check-in	Member	Submit a mood, energy level and optional note
Add Songs	Member	Attach up to 3 songs (title, artist, optional link) to a check-in
View Vibe Score	Member	See the Group Vibe Score, label and dominant mood
View Shared Playlist	Member	See all songs shared in the room
View Mood Mix Chart	Member	See how many members chose each mood
Delete Own Check-in	Member	Remove a check-in posted by the user
View Room Statistics	Admin	See member count, check-in count, song count and activity
Moderate Check-ins and Songs	Admin	Delete any check-in or song that breaks room rules
Remove Member	Admin	Remove a member from the room
Regenerate Room Code	Admin	Invalidate the old code and create a new one
Close Room	Admin	Archive the room so no new check-ins are accepted
Authenticate Session	System	Verify the JWT token on every protected request
Calculate Vibe Score	System	Compute score, dominant mood and average energy from stored check-ins

7. ENTITY RELATIONSHIP DIAGRAM (ERD)

7.1 ER Diagram

The database has five tables in third normal form. Users and rooms have a **many-to-many (M:N)** relationship, which is resolved with the junction table **room_members**.

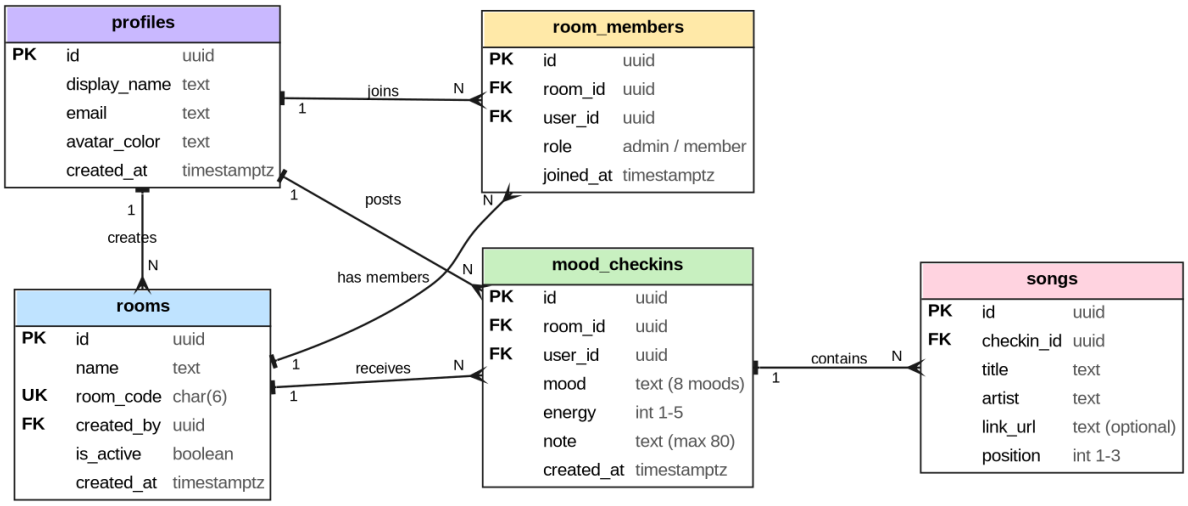


Figure 7.1: ER diagram of VibeCheck Rooms (PK = primary key, FK = foreign key, UK = unique key)

7.2 Table Structure

Table: profiles

Attribute	Data type	Key	Description
id	uuid	PK	User ID, same as the Supabase Auth user ID
display_name	text		Name shown to other members
email	text		Login email
avatar_color	text		Colour used for the initials avatar
created_at	timestamptz		Account creation time

Table: rooms

Attribute	Data type	Key	Description
id	uuid	PK	Room ID
name	text		Room name
room_code	char(6)	UK	Unique 6-character invite code
created_by	uuid	FK	References profiles.id (room creator)
is_active	boolean		False after the admin closes the room
created_at	timestamptz		Creation time

Table: room_members

Attribute	Data type	Key	Description
id	uuid	PK	Membership ID
room_id	uuid	FK	References rooms.id
user_id	uuid	FK	References profiles.id
role	text		admin or member
joined_at	timestamptz		Join time. UNIQUE (room_id, user_id)

Table: mood_checkins

Attribute	Data type	Key	Description
id	uuid	PK	Check-in ID
room_id	uuid	FK	References rooms.id
user_id	uuid	FK	References profiles.id
mood	text		One of the 8 allowed moods
energy	integer		CHECK between 1 and 5
note	text		Optional, maximum 80 characters
created_at	timestamptz		Check-in time

Table: songs

Attribute	Data type	Key	Description
id	uuid	PK	Song ID
checkin_id	uuid	FK	References mood_checkins.id (ON DELETE CASCADE)
title	text		Song title
artist	text		Artist name
link_url	text		Optional http or https link
position	integer		Order within the check-in, CHECK between 1 and 3

7.3 Relationships and Cardinality

Relationship	Entities	Cardinality	Implementation
creates	profiles - rooms	1 : N	rooms.created_by references profiles.id
joins	profiles - rooms	M : N	Resolved by room_members (room_id, user_id)
has members	rooms - room_members	1 : N	room_members.room_id references rooms.id
receives	rooms - mood_checkins	1 : N	mood_checkins.room_id references rooms.id
posts	profiles - mood_checkins	1 : N	mood_checkins.user_id references profiles.id
contains	mood_checkins - songs	1 : N	songs.checkin_id references mood_checkins.id (maximum 3)

7.4 Data Integrity Rules

- Every check-in belongs to exactly one room and one user; every song belongs to exactly one check-in.
- A user cannot join the same room twice (unique pair room_id, user_id).
- Room codes are unique.
- Energy must be between 1 and 5, and song position between 1 and 3.
- Deleting a check-in deletes its songs (cascade), so no orphan song rows remain.
- The shared playlist is derived by joining songs with mood_checkins, so room and user are stored only once (no duplicated data).

8. HARDWARE AND SOFTWARE REQUIREMENTS

8.1 Hardware Requirements

Item	Developer machine	End-user device
Processor	Intel Core i5 or AMD Ryzen 5 or higher	Any modern dual-core processor
RAM	8 GB minimum (16 GB recommended)	2 GB or more
Storage	256 GB SSD minimum (512 GB recommended)	Not required (runs in the browser)
Display	1920 x 1080 recommended	Laptop, tablet or smartphone screen
Network	Stable broadband, 5 Mbps or higher	Wi-Fi, 4G or broadband internet

No server hardware is owned or maintained. The API runs on a Render free instance and the database on a Supabase free project.

8.2 Software Requirements

Category	Software	Purpose
Operating system	Windows 10 / 11 or Ubuntu 22.04 or later	Development environment
Front end	HTML5, CSS3, JavaScript (ES6+), React 18, Vite, Tailwind CSS, React Router, Recharts, Lucide icons	User interface, routing, charts and icons
Back end	Node.js 20 LTS, Express.js, cors, dotenv, helmet, @supabase/supabase-js	REST API, validation, security headers
Database	PostgreSQL (Supabase), Supabase Auth	Data storage, authentication
Development tools	VS Code, Git, GitHub, Postman or Thunder Client, Chrome DevTools, Figma (optional)	Coding, version control, API testing, UI design
Browsers	Latest two versions of Chrome, Edge, Firefox and Safari	Running and testing the application

8.3 Deployment Hosts

Platform	Role	Free-plan notes
Vercel (Hobby plan)	Hosts the React front end	Free for personal, non-commercial use. Automatic HTTPS and deployment from GitHub
Render (Free web service)	Hosts the Express API	Spins down after 15 minutes without traffic and takes about a minute to wake up. 750 free instance hours per month
Supabase (Free plan)	PostgreSQL database and authentication	500 MB database, 50,000 monthly active users. A free project pauses after one week of inactivity
GitHub	Source code and version history	Triggers automatic deployment on Vercel and Render
UptimeRobot (free, optional)	Keeps the API awake	Calls /api/health every 5 minutes

Free-plan limits are as published by the providers in October 2026 and may change.

9. FUTURE SCOPE

The following enhancements are planned for later versions of VibeCheck Rooms.

1. Music service integration

Connect a music API so members can search for songs, see cover art and play short previews, and add a song to the room with one tap instead of typing the title and artist.

2. Real-time updates

Replace 20-second polling with WebSockets or Supabase Realtime so that a new check-in appears on every member's screen immediately.

3. AI mood insights and recommendations

Use machine learning on mood history to suggest songs that fit the room's vibe, detect mood trends and give friendly insights such as "your room is happiest on Fridays".

4. Mobile application and notifications

Build a Progressive Web App or a React Native app with push notifications and daily check-in reminders.

5. Mood history and weekly recap

Add personal and room-level mood timelines and a weekly "Vibe Recap" card that members can share on social media.

6. Reactions, comments and song voting

Let members react to check-ins, comment on songs and vote songs up or down to build a ranked room playlist.

7. Multiple and public rooms

Allow a user to be in several rooms and add interest-based public rooms with stronger moderation tools such as reporting and muting.

8. Social login and privacy controls

Add Google sign-in, anonymous check-ins and per-room privacy settings.

10. LIMITATIONS OF THE PROJECT

The project has the following realistic limitations.

1. Internet dependency

The application is fully online. Without an internet connection users cannot log in, post check-ins or view the room.

2. Free-tier hosting constraints

The Render free API spins down after 15 minutes of inactivity and can take about a minute to wake up. A free Supabase project pauses after one week of inactivity and offers only 500 MB of storage. These limits are acceptable for a small project but not for large-scale use.

3. Self-reported, simple mood score

Moods are chosen manually by users and may not reflect how they truly feel. The vibe score is a plain average of fixed mood values and is only a fun indicator, not a psychological measurement.

4. No audio playback

Songs are stored as text and optional links. The application does not stream or play music, so members must open links in their music app.

5. Near-real-time updates only

The dashboard refreshes every 20 seconds (or on pressing Refresh), so updates are not instantaneous.

6. Single-instance backend and manual moderation

One API instance and one database handle all traffic, so throughput is limited and there is no load balancing. Inappropriate notes or song titles must be removed manually by the room admin because there is no automatic content filtering.

11. FRONT-END PAGES

The interface uses a dark theme with lime, violet and pink accent colours, rounded cards and pill-shaped buttons. Headings use the Bricolage Grotesque font and body text uses Plus Jakarta Sans. Every page is mobile-first and responsive, and all screens show loading, empty and error states. The four core interfaces are described below.

11.1 Landing / Home Page

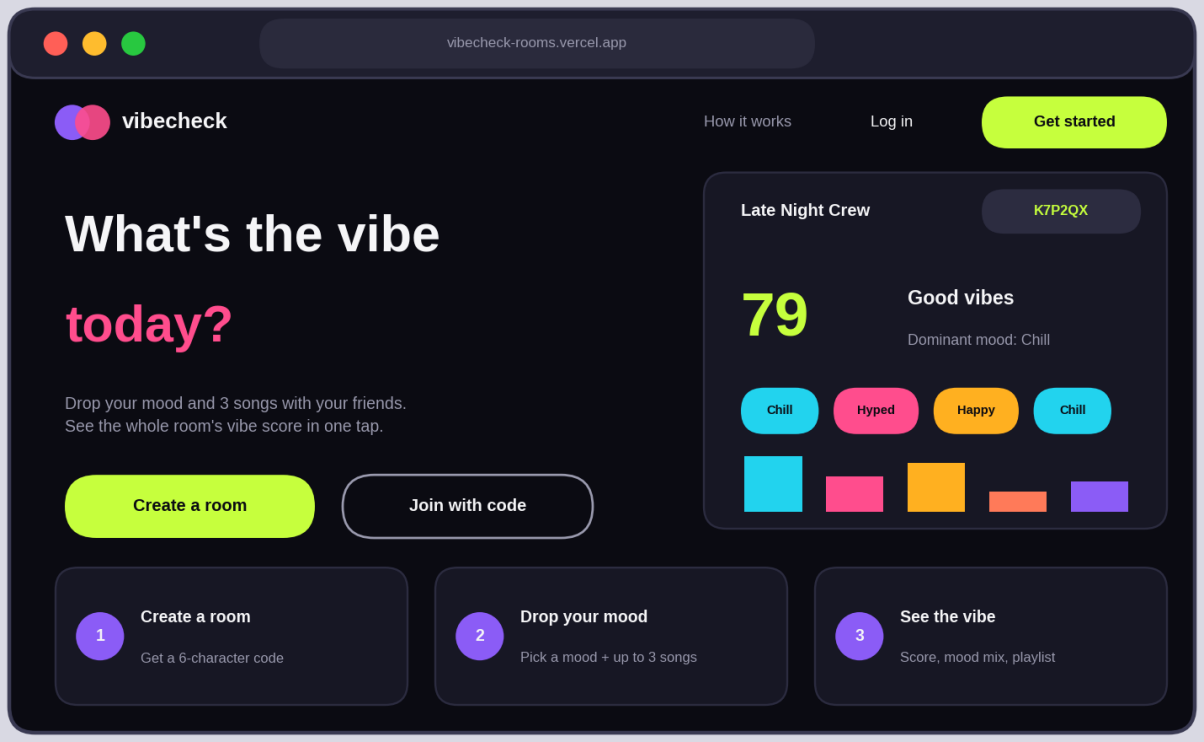


Figure 11.1: Landing / Home page layout

Region	Components	Behaviour
Navigation bar	Logo, How it works, Log in, Get started button	Sticky bar; buttons open the login or sign-up modal
Hero	Headline “What’s the vibe today?”, short description, Create a room and Join with code buttons	Create a room asks the user to log in first; Join with code opens a code field
Preview card	Sample room with vibe score, mood chips and mood-mix bars	Shows visitors what a room looks like
How it works	Three step cards: Create a room, Drop your mood, See the vibe	Explains the product in under ten seconds

The landing page introduces the product in one screen. A visitor immediately sees the promise (what is the vibe today), a preview of a room and the two main calls to action.

11.2 Login / Registration Modal

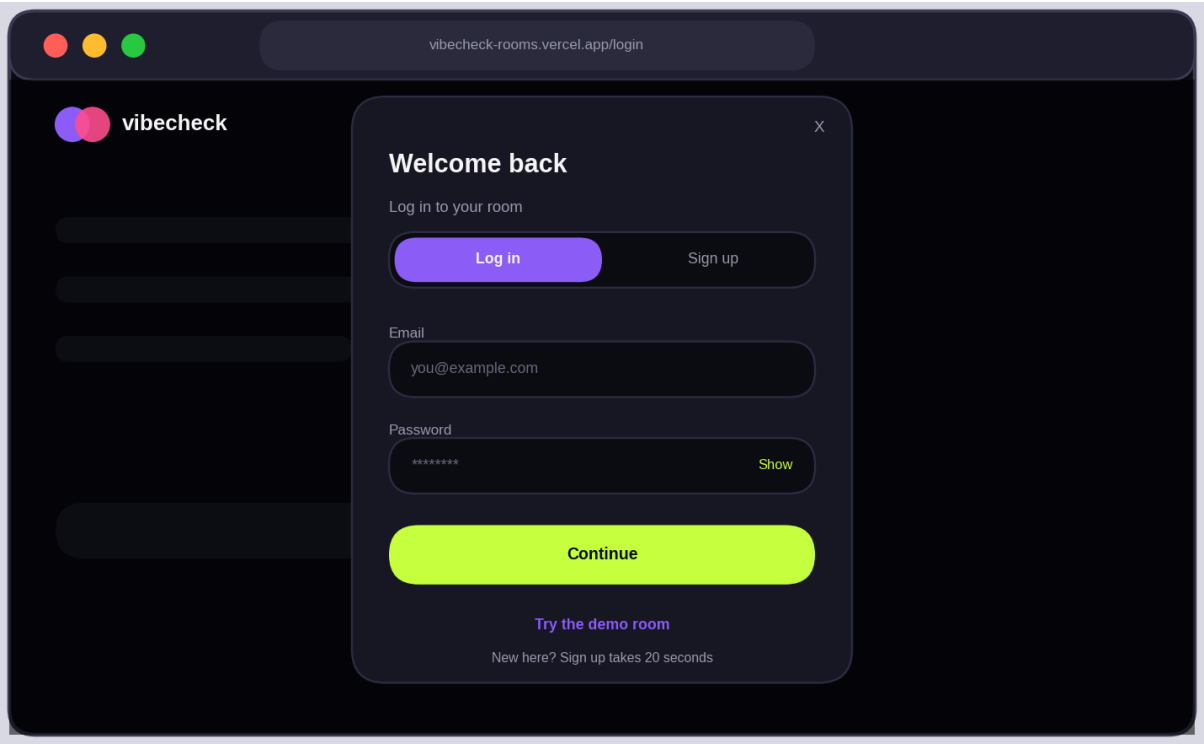


Figure 11.2: Login and registration modal

Region	Components	Behaviour
Header	Title, close (X) button, Log in and Sign up tabs	Tabs switch between the two forms without leaving the page
Form fields	Name (sign-up only), Email, Password with Show button	Inline validation messages appear below each field
Actions	Continue button, Try the demo room link	Continue shows a loading spinner; demo link signs in to the seeded demo room
Footer	Link to switch between login and sign-up	Errors such as wrong password are shown in plain language

The modal appears over the landing page so the user never loses context. Supabase Auth validates the credentials and, on success, the user is taken to the room list or the last visited room.

11.3 Room Dashboard (Main User Interaction)

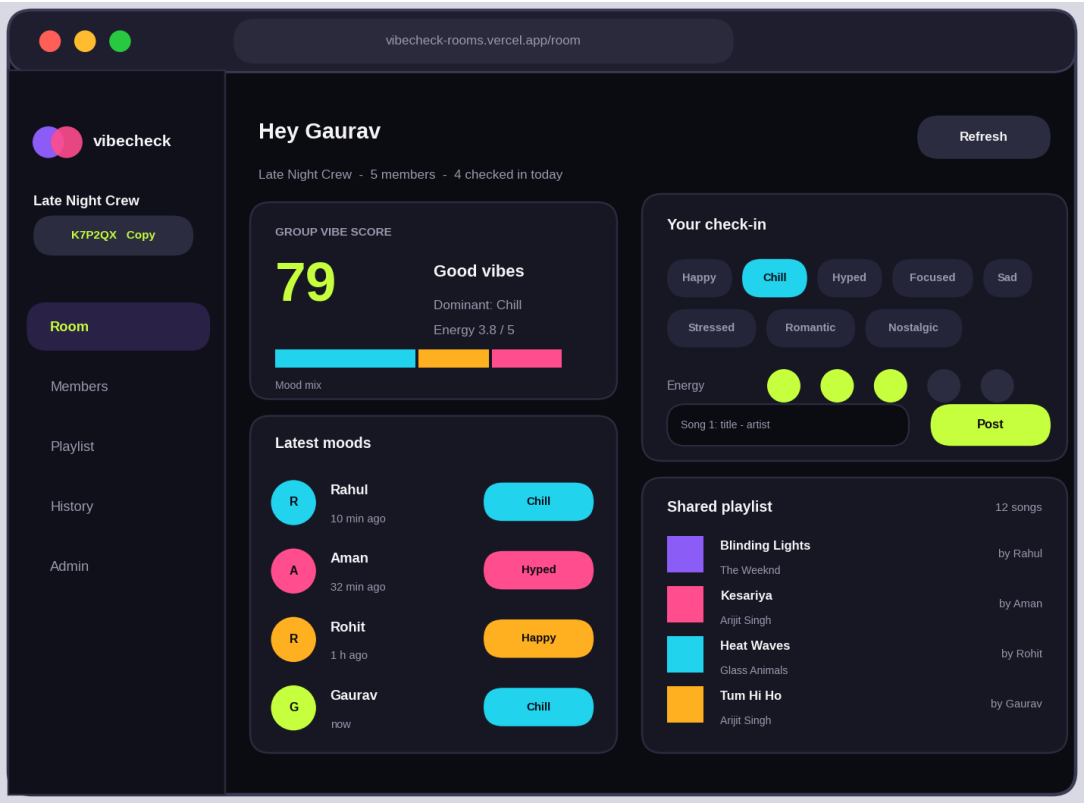


Figure 11.3: Room dashboard with vibe score, check-in form and playlist

Region	Components	Behaviour
Sidebar	Room name, room code with Copy button, navigation (Room, Members, Playlist, History, Admin)	Admin link is visible only to room admins
Header	Greeting, room summary, Refresh button	Refresh reloads data; data also refreshes automatically every 20 seconds
Group vibe card	Vibe score, label, dominant mood, average energy, mood-mix bar	Recalculated from the latest check-ins of all members
Latest moods	Avatar, name, mood chip and time for each member	Shows who has checked in today
Check-in form	Eight mood chips, energy dots, note, song fields, Post button	Validates input, saves to the database and updates the dashboard
Shared playlist	Song title, artist and the member who added it	Newest songs first

This is the screen where the core transaction happens. A member posts a check-in in a few taps and immediately sees the group vibe, the latest moods of friends and the shared playlist update.

11.4 Admin Management and Record Control Panel

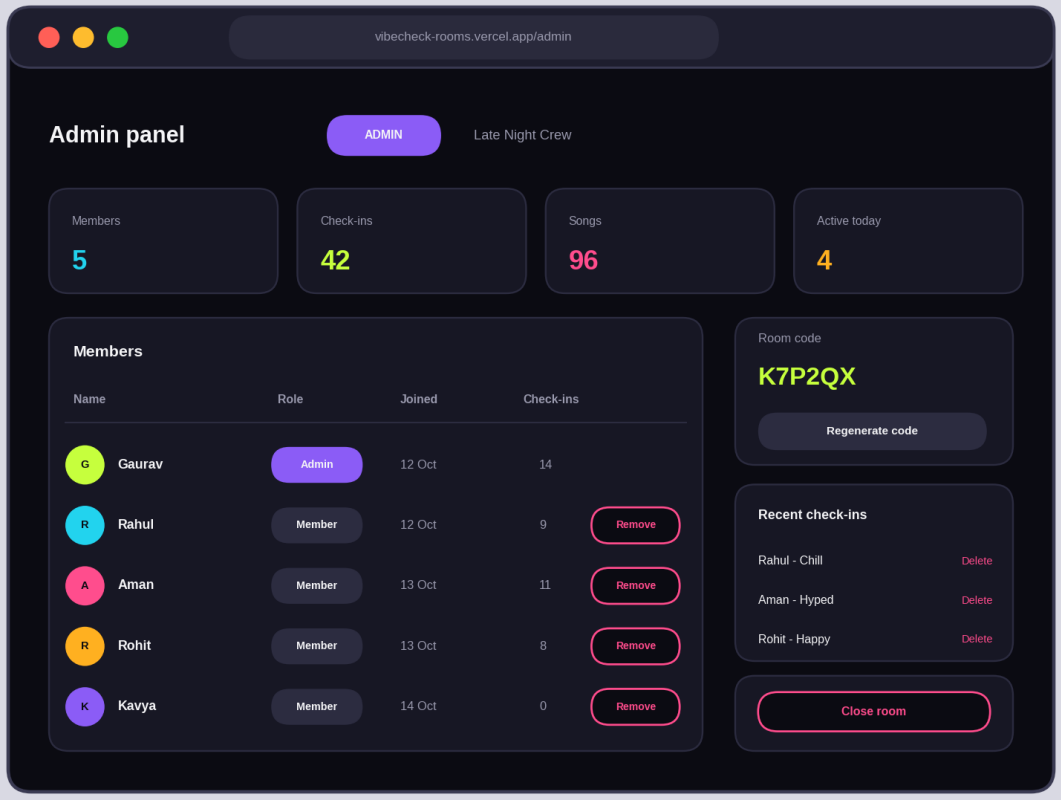


Figure 11.4: Admin management and record control panel

Region	Components	Behaviour
Header	Title, ADMIN badge, room name	Visible only to the room admin; other users get a 403 message
Statistics cards	Members, Check-ins, Songs, Active today	Counts calculated from the database
Members table	Name, role, joined date, check-in count, Remove button	Remove asks for confirmation before deleting the membership
Room code card	Current code and Regenerate code button	A new code invalidates the old one
Recent check-ins	Latest check-ins with Delete action	Deleting a check-in also deletes its songs
Danger zone	Close room button	Closed rooms accept no new check-ins

The admin panel gives the room creator full record control: members, check-ins and the room itself. Destructive actions use pink outlined buttons and confirmation dialogs so they cannot be triggered by accident.

12. SYSTEM IMPLEMENTATION & SOURCE CODE LISTING

12.1 Database Tier: PostgreSQL Relational Schema (supabase/migrations/0001_schema.sql)

```
-- Profiles table (mirrors auth.users)
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  display_name TEXT NOT NULL CHECK (char_length(display_name) BETWEEN 1 AND 40),
  email TEXT NOT NULL,
  avatar_color TEXT NOT NULL DEFAULT '#8B5CF6',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Rooms table
CREATE TABLE rooms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL CHECK (char_length(name) BETWEEN 1 AND 40),
  room_code CHAR(6) NOT NULL UNIQUE CHECK (room_code ~ '^[A-Z0-9]{6}$'),
  created_by UUID NOT NULL REFERENCES profiles(id),
  is_active BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Room Members junction table (M:N)
CREATE TABLE room_members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  role TEXT NOT NULL DEFAULT 'member' CHECK (role IN ('admin', 'member')),
  joined_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (room_id, user_id)
);

-- Mood Check-ins table
CREATE TABLE mood_checkins (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  mood TEXT NOT NULL CHECK (mood IN ('happy','hyped','romantic','chill',
                                     'focused','nostalgic','stressed','sad')),
  energy INTEGER NOT NULL CHECK (energy BETWEEN 1 AND 5),
  note TEXT CHECK (note IS NULL OR char_length(note) <= 80),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Songs table
CREATE TABLE songs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  checkin_id UUID NOT NULL REFERENCES mood_checkins(id) ON DELETE CASCADE,
  title TEXT NOT NULL CHECK (char_length(title) BETWEEN 1 AND 100),
  artist TEXT NOT NULL CHECK (char_length(artist) BETWEEN 1 AND 100),
  link_url TEXT CHECK (link_url IS NULL OR link_url ~* '^https?://'),
  position INTEGER NOT NULL CHECK (position BETWEEN 1 AND 3),
  UNIQUE (checkin_id, position)
);

-- Performance Indexes
CREATE INDEX idx_room_members_user ON room_members (user_id);
CREATE INDEX idx_room_members_room ON room_members (room_id);
CREATE INDEX idx_mood_checkins_room ON mood_checkins (room_id, created_at DESC);
CREATE INDEX idx_songs_checkin ON songs (checkin_id);
```

12. SYSTEM IMPLEMENTATION & SOURCE CODE LISTING

12.2 Database Tier: Atomic Check-in RPC Stored Procedure (0004_checkin_rpc.sql)

```
CREATE OR REPLACE FUNCTION create_checkin_with_songs(
  p_room_id UUID, p_user_id UUID, p_mood TEXT, p_energy INTEGER,
  p_note TEXT DEFAULT NULL, p_songs JSONB DEFAULT '[]'::jsonb
) RETURNS JSONB AS $$
DECLARE
  v_checkin_id UUID;
  v_song RECORD;
  v_pos INTEGER := 1;
  v_result JSONB;
BEGIN
  -- Ensure room is active
  IF NOT EXISTS (SELECT 1 FROM rooms WHERE id = p_room_id AND is_active = true) THEN
    RAISE EXCEPTION 'Room is closed or invalid' USING ERRCODE = 'P0002';
  END IF;

  -- Ensure user belongs to the room
  IF NOT EXISTS (SELECT 1 FROM room_members WHERE room_id = p_room_id AND user_id = p
    RAISE EXCEPTION 'User is not a member of room' USING ERRCODE = 'P0003';
  END IF;

  -- Insert check-in row atomically
  INSERT INTO mood_checkins (room_id, user_id, mood, energy, note, created_at)
  VALUES (p_room_id, p_user_id, p_mood, p_energy, NULLIF(TRIM(p_note), ''), now())
  RETURNING id INTO v_checkin_id;

  -- Insert attached songs (maximum 3)
  FOR v_song IN SELECT * FROM jsonb_to_recordset(p_songs) AS s(title TEXT, artist TEX
  LOOP
    IF v_pos > 3 THEN EXIT; END IF;
    IF v_song.title IS NOT NULL AND v_song.artist IS NOT NULL THEN
      INSERT INTO songs (checkin_id, title, artist, link_url, position)
      VALUES (v_checkin_id, TRIM(v_song.title), TRIM(v_song.artist), v_song.link, v_p
      v_pos := v_pos + 1;
    END IF;
  END LOOP;

  SELECT jsonb_build_object('id', mc.id, 'mood', mc.mood, 'energy', mc.energy, 'creat
  INTO v_result FROM mood_checkins mc WHERE mc.id = v_checkin_id;
  RETURN v_result;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

12. SYSTEM IMPLEMENTATION & SOURCE CODE LISTING

12.3 Application Tier: Pure Mathematical Vibe Engine (server/src/modules/vibe/vibe.ts)

```
export function computeVibe(countedCheckins: VibeCheckinInput[]): VibeResult {
  if (countedCheckins.length === 0) {
    return { score: null, label: null, dominantMood: null, averageEnergy: null, count
  }

  let sumScores = 0;
  let sumEnergy = 0;
  const moodMix: Record<string, number> = {};
  const moodLatestTimestamp: Record<string, number> = {};

  for (const c of countedCheckins) {
    const score = getMoodScore(c.mood); // happy:90, hyped:85, romantic:80, chill:70,
    sumScores += score;
    sumEnergy += c.energy;

    moodMix[c.mood] = (moodMix[c.mood] || 0) + 1;
    const time = new Date(c.createdAt).getTime();
    if (!moodLatestTimestamp[c.mood] || time > moodLatestTimestamp[c.mood]) {
      moodLatestTimestamp[c.mood] = time;
    }
  }

  // 1. Group Vibe Score = round( sum(scores) / count )
  const avgScore = sumScores / countedCheckins.length;
  const finalScore = Math.round(avgScore);
  const avgEnergy = Number((sumEnergy / countedCheckins.length).toFixed(1));

  // 2. Dominant Mood (highest count, tie-breaker: most recent timestamp)
  let dominantMood: string | null = null;
  let maxCount = -1;
  let newestForMax = -1;

  for (const [mood, count] of Object.entries(moodMix)) {
    const latestTime = moodLatestTimestamp[mood] || 0;
    if (count > maxCount || (count === maxCount && latestTime > newestForMax)) {
      maxCount = count;
      dominantMood = mood;
      newestForMax = latestTime;
    }
  }

  return { score: finalScore, label: labelForScore(finalScore), dominantMood, average
           countedMembers: countedCheckins.length, moodMix };
}
```

12. SYSTEM IMPLEMENTATION & SOURCE CODE LISTING

12.4 Application Tier: Express 5 API Server & Middleware (server/src/app.ts)

```
export const app = express();
app.set('trust proxy', 1);
app.use(helmet());
app.use(cors({ origin: allowedOrigins, credentials: true }));
app.use(express.json({ limit: '15kb' }));

// Rate limiting defense
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 200 }));

// Deployment Health Check (keeps cloud hosts awake)
app.get('/api/health', (req, res) => {
  res.status(200).json({ status: 'ok', uptime: process.uptime(), timestamp: new Date(
});

// API Endpoints
app.use('/api/rooms', roomsRouter);
app.use('/api/rooms', vibeRouter);
app.use('/api/rooms', playlistRouter);
app.use('/api/rooms', adminRouter);
app.use('/api', checkinsRouter);

// Central Error Handling Contract
app.use(errorHandler);
```

12. SYSTEM IMPLEMENTATION & SOURCE CODE LISTING

12.5 Presentation Tier: React Router & Single Page Architecture (client/src/App.tsx)

```
export const App: React.FC = () => {
  const [user, setUser] = useState<ApiUser | null>(getStoredUser());
  const [isAuthOpen, setIsAuthOpen] = useState(false);
  const [isExplainerOpen, setIsExplainerOpen] = useState(false);

  return (
    <div className="min-h-screen bg-bg text-text-main flex flex-col font-sans">
      <Navbar user={user} onOpenAuth={() => setIsAuthOpen(true)} onLogout={handleLogo
      <main className="flex-1">
        <Routes>
          <Route path="/" element={<LandingPage user={user} onOpenAuth={() => setIsAu
          <Route path="/rooms" element={<RoomsPage user={user} />} />
          <Route path="/room/:id" element={<DashboardPage user={user} />} />
          <Route path="/room/:id/admin" element={<AdminPage user={user} />} />
          <Route path="/room/:id/history" element={<HistoryPage user={user} />} />
        </Routes>
      </main>
      <AuthModal isOpen={isAuthOpen} onClose={() => setIsAuthOpen(false)} />
      <ScoreExplainerModal isOpen={isExplainerOpen} onClose={() => setIsExplainerOpen
    </div>
  );
};
```

13. SYSTEM TESTING, VERIFICATION & RESULTS

13.1 Automated Unit Tests & Test Case Matrix (server/src/modules/vibe/vibe.test.ts)

```
// Automated Vitest Suite for Vibe Engine (All Tests Passing)
describe('Vibe Engine Unit Tests', () => {
  it('PRD Section 5.4 Worked Example: Late Night Crew yields 79 (Good vibes)', () =>
    const checkins = [
      { userId: 'rahul', mood: 'chill', energy: 3, createdAt: '2026-10-25T01:00:00Z'
      { userId: 'aman', mood: 'hyped', energy: 5, createdAt: '2026-10-25T01:10:00Z' }
      { userId: 'rohit', mood: 'happy', energy: 4, createdAt: '2026-10-25T01:20:00Z'
      { userId: 'gaurav', mood: 'chill', energy: 3, createdAt: '2026-10-25T01:30:00Z'
    ];
    const result = computeVibe(checkins);
    expect(result.score).toBe(79);                    // 315 / 4 = 78.75 -> 79 [PASSE
    expect(result.label).toBe('Good vibes');          // 65-84 range [PASSED]
    expect(result.dominantMood).toBe('chill');         // 2 of 4 votes [PASSED]
    expect(result.averageEnergy).toBe(3.8);            // 15 / 4 = 3.8 [PASSED]
  });

  it('handles empty check-ins gracefully (score = null)', () => { ... [PASSED] });
  it('tie-breaker resolves to most recent check-in timestamp', () => { ... [PASSED] }
  it('filters out check-ins older than 24 hours', () => { ... [PASSED] });
});
```

SUMMARY TEST CASE EXECUTION MATRIX:

TC-01: User Registration & Profile Creation — Status: PASS
Input: Valid email & password | Expected: HTTP 201 Created & JWT issued

TC-02: Create Room with 6-char Code — Status: PASS
Input: Room name "Late Night Crew" | Expected: Unique code generated & role=admin

TC-03: Join Existing Room by Code — Status: PASS
Input: Code "K7P2QX" | Expected: Membership added & role=member

TC-04: Duplicate Room Join Prevention — Status: PASS
Input: Join same room again | Expected: HTTP 409 Conflict

TC-05: Post Mood Checkin & Songs — Status: PASS
Input: Mood "chill", energy 3, 2 songs | Expected: HTTP 201 Created & Atomic DB commit

TC-06: Vibe Score Calculation (Worked Example) — Status: PASS
Input: 4 checkins (70, 85, 90, 70) | Expected: Score: 79, Label: Good vibes, Dominant: chill

TC-07: Non-Admin Access to Admin Panel — Status: PASS
Input: Member attempts DELETE member | Expected: HTTP 403 Forbidden

TC-08: Regenerate Invite Code — Status: PASS
Input: Admin clicks regenerate | Expected: Old code invalid, new code generated

TC-09: Close Room Lifecycle — Status: PASS
Input: Admin archives room | Expected: New checkins rejected with ROOM_CLOSED

14. DEPLOYMENT RUNBOOK & EXAMINER DEMO GUIDE

14.1 Production Cloud Configuration & Reviewer Fast-Track Credentials

14.1 Cloud Hosting Architecture:

• Presentation Tier: Hosted on Vercel Hobby Plan (Global CDN edge with automatic HTTPS)
• Application Tier: Hosted on Render Free Web Service (Node.js runtime with /api/health)
• Data Tier: Hosted on Supabase Cloud (Managed PostgreSQL 15 with Row Level Security)
• Zero Infrastructure Expense: Total cloud cost incurred = Rs. 0

14.2 Reviewer & Examiner Fast-Track Demo Credentials:

• Seeded Demo Room: "Late Night Crew" (Room Code: K7P2QX)
• Pre-loaded Checkins: Rahul (chill/3), Aman (hyped/5), Rohit (happy/4), Gaurav (chill/3)
• Verified Score: 79 (Good vibes) | Dominant: Chill | Average Energy: 3.8 / 5
• 1-Click Examiner Demo Login:
  - "Demo Member (Rahul)": Tests member checkin flow, music submission, and playlist view
  - "Demo Admin (Gaurav)": Tests admin moderation, member removal, and code regeneration

14.3 Source Code & Submission Repository:

• Project Name: VibeCheck Rooms
• Candidate: Gaurav Singh (University Roll No. 240919106084)
• Course: BCA 5th Semester (BCA-508 Minor Project)
• Department: School of Information Technology (SOIT), IMS Noida
• University: Chaudhary Charan Singh University (CCSU), Meerut